

**Exercice 1 :****Problème de pleins de carburant**

On souhaite faire un certain trajet (prédéterminé) en voiture, en partant avec un réservoir plein et en remplissant le réservoir à l'arrivée. Après un plein de carburant, on peut rouler une distance  $m$  qui est supposée constante.

On se donne  $(d_0, \dots, d_n)$  un  $n+1$ -uplet représentant les abscisses des stations-services sur le trajet.  $d_0 = 0$  correspond au point de départ, et  $d_n$  le point d'arrivée. Pour tout  $i \in \llbracket 0; n-1 \rrbracket$ ,  $d_i < d_{i+1} \leq d_i + m$ , en d'autres termes il est possible d'aller de la station service d'indice  $i$  à celle d'indice  $i+1$  sans tomber en panne.

On souhaite déterminer un tuple strictement croissant  $(i_1, \dots, i_k) \in (\mathbb{N}^*)^{(N)}$  tel que, en posant  $i_0 = 0$ , on ait :

$$\begin{cases} i_k = n \\ \forall j \in \llbracket 0; k-1 \rrbracket, d_{i_{j+1}} \leq d_{i_j} + m \end{cases}$$

Un tel tuple correspond à un choix valide d'indice de stations service (plus le point de départ) où on s'arrêtera sur le chemin.

On souhaite de plus choisir un tuple qui minimise le nombre d'arrêt  $k$ . On dit qu'un  $k$ -uplet  $(i_1, \dots, i_k)$  est *solution* du problème de paramètres  $(m, (d_0, \dots, d_n))$  s'il satisfait les contraintes, et qu'il est *solution optimale* si de plus aucun  $k'$ -uplet avec  $k' < k$  arrêts n'est solution.

Par convention, le problème où le point de départ est égal au point d'arrivée (c'est à dire  $n = 0$ ) a une unique solution qui est le tuple vide<sup>1</sup>.

1. On dit que le *choix glouton* sur ce problème consiste à rouler jusqu'à la station service la plus lointaine qu'on peut atteindre depuis le départ.

Un *algorithme glouton* consiste à voir le problème d'arriver à  $d_n$  depuis une station service où on vient de faire le plein comme un problème de la même forme (mais plus simple) que le problème d'origine, et donc à résoudre le problème récursivement en faisant à chaque étape le choix glouton.

Écrire une fonction `arrets_glouton : int -> int array -> int list` qui prend en argument  $m$  et le tableau de longueur  $n+1$  `[d0; ...; dn]`, et renvoie la liste des distances des stations où on s'arrêtera `[di1; ...; di_k]`.

Tester avec  $m = 20$  et les distances des stations-services  $(0, 6, 14, 23, 32, 43, 48, 60, 67, 76, 86, 95, 110)$ .

2. On souhaite maintenant montrer la correction de notre algorithme :

- (a) Montrer que le problème dispose d'une *sous-structure*, c'est à dire que pour tout  $n \geq 1$ , si  $(i_1, \dots, i_k)$  avec  $k \geq 1$  vérifie :  $\begin{cases} d_{i_1} \leq m \\ (i_2, \dots, i_k) \text{ est solution du problème de paramètres } (m, (d'_0, d'_1, \dots, d'_{n-i_1})) \end{cases}$  avec  $\forall i \in \llbracket 0; n-i_1 \rrbracket, d'_i = d_{i+i_1} - d_{i_1}$ , alors  $(i_1, \dots, i_k)$  est solution du problème de paramètres  $(m, (d_0, \dots, d_n))$ .
- (b) Montrer que la sous-structure est *optimale*, c'est à dire que si  $(i_1, \dots, i_k)$  est une solution optimale du problème de paramètres  $(m, (d_0, \dots, d_n))$  pour  $n \geq 1$ , alors  $(i_2, \dots, i_k)$  est une solution optimale du problème de paramètres  $(m, (d'_0, d'_1, \dots, d'_{n-i_1}))$ .
- (c) Montrer que le *choix glouton est toujours possible*, c'est à dire que si  $(i_1, \dots, i_k)$ , avec  $k \geq 2$  est une solution du problème, et que  $i'_1$  est le choix glouton pour le premier arrêt, il existe  $j \in \llbracket 2; k \rrbracket$  tel que  $(i'_1, i_j, i_{j+1}, i_{j+2}, \dots, i_k)$  est toujours une solution du problème.

**Remarque :** On a posé  $k \geq 2$ , car on n'a pas de choix pour  $i_k$  qui doit nécessairement être  $n$ . Si il existe un 1-uplet solution, c'est  $(n)$  et c'est exactement ce que donne l'algorithme glouton.

- (d) Dédurre des propriétés ci-dessus une preuve par récurrence que `arrets_gloutons` renvoie bien la solution optimale du problème.

1. On n'a pas besoin de s'arrêter faire le plein à l'arrivée, puisque le réservoir est déjà plein et qu'on n'a pas roulé.

**Exercice 2** :**Rendu de monnaie**

On se donne un *système de pièces et billets* sous la forme d'un tuple d'entiers  $(v_0, \dots, v_{n-1})$ , non vide et strictement croissant, qui vérifie  $v_0 = 1$ .

Le problème du rendu de monnaie sur ce système, consiste, étant donné un entier  $N \in \mathbb{N}$ , à trouver un tuple  $(i_1, \dots, i_k) \in \llbracket 0; n-1 \rrbracket^{(\mathbb{N})}$  tel que  $N = \sum_{j=1}^k v_{i_j}$ .

Une solution du problème du rendu de monnaie est optimale si elle minimise  $k$ .

Le choix glouton pour ce problème consiste à choisir pour  $i_1$  le plus grand  $i$  tel que  $v_i \leq N$ , en d'autres termes commencer par rendre la plus grande dénomination qui ne dépasse pas le montant à rendre.

1. Écrire une fonction récursive `rendu_glouton : int array -> int -> int list` qui prend en argument le tableau  $[|v_0; \dots; v_{n-1}|]$ , supposé strictement croissant qui indique les valeurs des pièces et billets disponibles, et  $N$  une somme à rendre, et renvoie la liste des valeurs rendues par le choix glouton.
2. Montrer que pour tout système, le problème possède une sous-structure optimale.
3. Montrer que pour le système  $(1, 2, 5, 10)$ , qui est une partie du système en euros<sup>2</sup>, le choix glouton est toujours possible.

En déduire la preuve de la correction de l'algorithme glouton pour ce système.

**Remarque :** Un système qui vérifie cette propriété est dit *canonique*. Le système en dollars  $(1, 5, 10, 25, 100, 500, 1000, 2000, 10000)$  est également canonique.

4. Montrer que le système suivant (système britannique avant la réforme de décimalisation en 1971<sup>3</sup>, dit "pence/shilling/livre" ou "p/d/£") n'est pas canonique :  $(1, 2, 4, 6, 12, 24, 48, 60, 120, 240, 480, 960, 2400)$

2. La preuve pour le système complet  $(1, 2, 5, 10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000, 20000)$ , représentant les valeurs en centimes des pièces et billets ayant cours légal, ne présente pas plus de difficulté mais est laborieuse.

3. Les valeurs relatives des pièces n'ont pas bougé depuis le 18ème siècle, mais les pièces en circulations ne sont plus les mêmes. Le système présenté est celui qui avait cours entre 1961 et 1969 uniquement, les halfpennies et les demi-couronnes ayant été éliminés dès 1970, les farthings (quart de pence) après 1960. Cependant, rajouter les farthings, ou les autres pièces ayant eu cours au 19ème ou 20ème siècle, ne change pas le caractère non canonique du système. Les valeurs sont données en halfpennies. Les noms des pièces sont, dans l'ordre : halfpenny, penny, twopence, threepence, sixpence, shilling, florin, half-crown, crown, half sovereign, sovereign, two pounds, five pounds.